

Concurrency

Introduction 🎬

Learn the fundamentals of concurrency and when it shows up in low-level design interviews.

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

▶ Watch Now

Concurrency is what happens when multiple things try to happen at the same time. Two users book the same flight seat. Three threads update the same counter. A dozen requests hit your cache while it's mid-refresh. The code that worked perfectly in testing suddenly produces impossible results in production.

Concurrency doesn't necessarily show up in every low-level design interview. It depends on the company, the team, and the level. But once you interview for senior roles, it becomes common to either show up directly or get added as a follow-up.



Low-level design interviews focus on single-process systems, so the concurrency here is all threads and shared memory within one program. System design interviews deal with concurrency across multiple servers — that's a different beast. If you're looking for that, check out [Dealing with Contention](#).

In interviews, concurrency shows up in two ways. Sometimes a classic LLD question gets harder: a parking lot that worked fine now has two cars racing for the same spot, or an inventory system where two orders fight over the last item. Other times the prompt is built around concurrency from the start: thread pools, rate limiters, connection pools, schedulers. Either way, the interviewer wants to see if you can reason about what breaks when actions overlap and fix it without overcomplicating things.

This article covers the mental models you need. We'll start with the fundamentals, introduce the primitives you'll reach for, and then map out the three categories of concurrency problems you'll encounter in interviews and how to solve them.

Concurrency Fundamentals

Concurrency starts with a basic fact: threads in the same process share memory.

When a program runs, the operating system creates a process—an isolated container with its own address space and resources. Inside that process, the OS or language runtime can create one or more threads. A thread is an independent execution path. It has its own program counter, registers, and stack, but it shares the heap, globals, and open resources with other threads in the same process.

Concurrency exists whenever multiple threads can make progress independently and their execution can overlap. On multi-core machines, threads may run in parallel. On a single core, the OS rapidly switches between threads and interleaves their instructions. From the program's point of view, both cases are the same: operations from different threads can interleave in unpredictable ways.

That unpredictability is the root of concurrency bugs. Code that looks atomic at the source level is often multiple machine instructions. If two threads read and write shared memory without coordination, the outcome can depend on timing, scheduling, or load. This is why concurrency bugs are often nondeterministic and hard to reproduce.

Most production languages and systems live in this multi-threaded world by default. Java, C++, Go, Rust, C#, and Python all run code concurrently in real systems. That means concurrency should be assumed any time shared state exists.



JavaScript and TypeScript are notable exceptions because user code runs on a single main thread, with concurrency expressed through event loops and async callbacks rather than shared-memory threads.

The Toolbox

Concurrency problems are solved with a small set of primitives that every language provides in some form. You don't need to invent new synchronization mechanisms, you only need to recognize which existing tool fits the problem and how to apply it. The primitives below handle the vast majority of interview scenarios.



Already comfortable with concurrency primitives? Skip ahead to [Three Problem Types](#) to see how they map to interview scenarios. Each primitive is covered in depth in [Correctness](#), [Coordination](#), and [Scarcity](#). This section just serves as a quick reference.

Atomics

Atomics provide thread-safe operations on single variables without locks. Under the hood, they use CPU instructions like compare-and-swap (CAS) that complete in one uninterruptible step.

Python is a corner case that lacks native atomics, so you'll need locks to safely increment counters across threads. The example below shows using `threading.Lock` to protect a shared counter.

atomics.py

Python

```
import threading

lock = threading.Lock()
counter = 0

with lock:
```

Use atomics for counters, flags, and simple statistics. They're fast but limited to single variables, so the moment you need to update two things together, atomics no longer help.

Used in: [Correctness](#) for read-modify-write on single variables.

Locks (Mutexes)

Locks provide mutual exclusion. When a thread holds a lock, other threads trying to acquire it block until the first thread releases. This creates a critical section where only one thread executes at a time.

locks.py

Python

```
import threading

lock = threading.Lock()
```

```
with lock:
    # Only one thread can be here at a time
    balance += amount
```

Locks are your default tool for protecting shared state. The main variants are coarse-grained (one lock for everything), fine-grained (per-resource locks), and read-write (multiple readers OR one writer).

Used in: **Correctness** for check-then-act and multi-field updates.

Semaphores

Semaphores are counting locks. Instead of binary locked/unlocked, a semaphore has N permits. Threads acquire permits before proceeding and release them when done. When permits hit zero, threads block until someone releases.

 semaphores.py

Python  

```
import threading

permits = threading.Semaphore(5) # Allow 5 concurrent operations
permits.acquire() # Block if no permits available
try:
    do_work()
finally:
    permits.release() # Always release, even on exception
```

Use semaphores to limit concurrent operations, like when you can have at most 5 downloads or at most 10 API calls at a given time.

Used in: **Scarcity** for limiting concurrent operations and resource budgets.

Condition Variables

Condition variables let threads wait efficiently for a condition to become true. A thread acquires a lock, checks a condition, and if not satisfied, waits. This atomically releases the lock

and puts the thread to sleep. When another thread signals, waiters wake up and re-check.



condition_variables.py

Python



```
import threading

condition = threading.Condition()

with condition:
    while not ready:
        condition.wait() # Release lock and sleep
    # Condition is now true
```

Condition variables are the building block for blocking queues, but you rarely use them directly in interviews.

Used in: **Coordination** as the foundation for blocking queues.

Blocking Queues

Blocking queues combine a queue with condition variables to provide thread-safe producer-consumer handoff. Producers call `put()` to add items; if full, they block. Consumers call `take()` to remove items; if empty, they block.



blocking_queues.py

Python



```
import queue

q = queue.Queue(maxsize=100)
q.put(task) # Blocks if queue is full
t = q.get() # Blocks if queue is empty
```

The queue handles all synchronization internally making it your go-to tool for handing work between threads.

Used in: **Coordination** for producer-consumer and async processing. Also in **Scarcity** for resource pooling.

Language Reference

Here is a quick reference of concurrency primitives across common interview languages.

Concept	Java	Python	Go
Lock/Mutex	<code>synchronized</code> / <code>ReentrantLock</code>	<code>threading.Lock</code>	<code>sync.Mutex</code>
Read-write lock	<code>ReentrantReadWriteLock</code>	N/A (3rd party)	<code>sync.RWMutex</code>
Condition variable	<code>Object.wait/notify</code>	<code>threading.Condition</code>	<code>sync.Cond</code>
Semaphore	<code>Semaphore</code>	<code>threading.Semaphore</code>	<code>x/sync/semaphore</code>
Blocking queue	<code>LinkedBlockingQueue</code>	<code>queue.Queue</code>	buffered channel
Atomic integer	<code>AtomicInteger</code>	N/A (use Lock)	<code>sync/atomic</code>
Concurrent map	<code>ConcurrentHashMap</code>	N/A (GIL)	<code>sync.Map</code>

Notes:

- Python's GIL means CPU-bound code doesn't benefit from threads, but I/O-bound code does. Use `multiprocessing` for CPU parallelism.
- Go channels replace blocking queues and condition variables idiomatically. When in doubt, use a channel.
- C++ often requires manual composition of primitives. Consider Intel TBB for higher-level abstractions.

Three Problem Types

Most concurrency problems in interviews fall into three categories. The surface details change - inventory systems, booking systems, rate limiters - but the underlying failure modes don't. Learning to see past the domain and into the problem type is the skill we'll cover in this article.

Correctness problems happen when shared state gets corrupted. Two threads both check that a seat is available, both see yes, both book it. One booking gets lost.

Coordination problems happen when threads need to hand off work or wait for each other. A producer adds tasks to a queue, consumers process them. If the queue is empty, consumers need to wait efficiently without burning CPU. If the queue is full, producers need to slow down.

Scarcity problems happen when resources are limited. You have 10 database connections but 100 concurrent requests. Some requests must wait.

Problem Type	What Breaks	Solutions	Common Problems
<u>Correctness</u>	Shared state is updated concurrently	Locks, atomics, thread confinement	Check-then-act, read-modify-write
<u>Coordination</u>	Threads need ordering or handoff	Blocking queues, actors, event loops	Async request processing, bursty traffic
<u>Scarcity</u>	Resources are limited	Semaphores, resource pools	Concurrent op limits, resource consumption, object reuse

Most questions start with correctness. Coordination and scarcity often appear as follow-ups once shared state exists or throughput increases. Real systems frequently involve more than one category, but separating them makes it easier to reason about each concern in isolation.

What's Next

The articles that follow dive into each problem category in turn. Each explains what breaks, why it breaks, and which primitives to reach for. The goal is establish pattern recognition so you can diagnose concurrency risks early, pick the right tool, and explain your reasoning clearly.

Start with **Correctness** if you want to understand how shared state gets corrupted and how locks and atomics prevent it. Move to **Coordination** for producer-consumer patterns and async

processing, then finish with Scarcity for resource pooling and rate limiting.